Generation as Computation

Generalised Computation in the Set-theoretic Universe and Beyond
(Thesis Proposal)

Desmond Lau

National University of Singapore

November 20, 2025

Outline

1. Introduction
   - Background: Infinitary Computation
   - Background: Formulas as Programs
   - Overview
2. Generalised Computation Within V
   - Generalised Sequential Algorithms
   - Comparisons with Other Notions
3. Generalised Computation Beyond V
   - Degrees of Small Extensions
   - Local Method Definitions

Outline

Gödel's Informal Proposition

- In an essay Gödel wrote in 1958 (but only published posthumously) [4], he criticised Hilbert's choice of "intuitively clear" notions meant to make up the bedrock of classical mathematics.
- He lamented that "the domain of 'finitary mathematics"' is unnaturally restrictive.
- In the same work, Gödel informally described a notion of infinitary computation built inductively from natural numbers.
- He called this notion *computable functions of finite type*.

Gödel's Informal Proposition

- In an essay Gödel wrote in 1958 (but only published posthumously) [4], he criticised Hilbert's choice of "intuitively clear" notions meant to make up the bedrock of classical mathematics.
- He lamented that "the domain of 'finitary mathematics" is unnaturally restrictive.
- In the same work, Gödel informally described a notion of infinitary computation built inductively from natural numbers.
- He called this notion *computable functions of finite type*.

Gödel's Informal Proposition

- In an essay Gödel wrote in 1958 (but only published posthumously) [4], he criticised Hilbert's choice of "intuitively clear" notions meant to make up the bedrock of classical mathematics.
- He lamented that "the domain of 'finitary mathematics"' is unnaturally restrictive.
- In the same work, Gödel informally described a notion of infinitary computation built inductively from natural numbers.
- He called this notion *computable functions of finite type*.

Gödel's Informal Proposition

- In an essay Gödel wrote in 1958 (but only published posthumously) [4], he criticised Hilbert's choice of "intuitively clear" notions meant to make up the bedrock of classical mathematics.
- He lamented that "the domain of 'finitary mathematics"' is unnaturally restrictive.
- In the same work, Gödel informally described a notion of infinitary computation built inductively from natural numbers.
- He called this notion *computable functions of finite type.*

Gödel's Informal Proposition

- In an essay Gödel wrote in 1958 (but only published posthumously) [4], he criticised Hilbert's choice of "intuitively clear" notions meant to make up the bedrock of classical mathematics.
- He lamented that "the domain of 'finitary mathematics"' is unnaturally restrictive.
- In the same work, Gödel informally described a notion of infinitary computation built inductively from natural numbers.
- He called this notion *computable functions of finite type*.

More Formal and Explicit Conceptions

- The eponymous Blum-Shub-Smale machines [5], born from the study of dynamical systems.
- Infinite-time Turing machines [6] by Hamkins and Lewis, motivated by natural philosophical questions.
- More recently, MS-machines [7] by Yang *et al.*

More Formal and Explicit Conceptions

- The eponymous Blum-Shub-Smale machines [5], born from the study of dynamical systems.
- Infinite-time Turing machines [6] by Hamkins and Lewis, motivated by natural philosophical questions.
- More recently, MS-machines [7] by Yang *et al.*

More Formal and Explicit Conceptions

- The eponymous Blum-Shub-Smale machines [5], born from the study of dynamical systems.
- Infinite-time Turing machines [6] by Hamkins and Lewis, motivated by natural philosophical questions.
- More recently, MS-machines [7] by Yang *et al.*

More Formal and Explicit Conceptions

- The eponymous Blum-Shub-Smale machines [5], born from the study of dynamical systems.
- Infinite-time Turing machines [6] by Hamkins and Lewis, motivated by natural philosophical questions.
- More recently, MS-machines [7] by Yang *et al*.

Outline

1. Introduction
   - Background: Infinitary Computation
   - Background: Formulas as Programs
   - Overview
2. Generalised Computation Within V
   - Generalised Sequential Algorithms
   - Comparisons with Other Notions
3. Generalised Computation Beyond V
   - Degrees of Small Extensions
   - Local Method Definitions

In Classical Computability Theory

- "A program is a formula (of a particular form)" — an oft-repeated slogan.
- An oracle machine can functionally be represented as a sufficiently simple parametrised formula in the language of arithmetic.
- This is a philosophical upshot of relating the arithmetical hierarchy to certain ideals of the Turing degrees.

In Classical Computability Theory

- "A program is a formula (of a particular form)" — an oft-repeated slogan.
- An oracle machine can functionally be represented as a sufficiently simple parametrised formula in the language of arithmetic.
- This is a philosophical upshot of relating the arithmetical hierarchy to certain ideals of the Turing degrees.

In Classical Computability Theory

- "A program is a formula (of a particular form)" — an oft-repeated slogan.
- An oracle machine can functionally be represented as a sufficiently simple parametrised formula in the language of arithmetic.
- This is a philosophical upshot of relating the arithmetical hierarchy to certain ideals of the Turing degrees.

In Classical Computability Theory

- "A program is a formula (of a particular form)" — an oft-repeated slogan.
- An oracle machine can functionally be represented as a sufficiently simple parametrised formula in the language of arithmetic.
- This is a philosophical upshot of relating the arithmetical hierarchy to certain ideals of the Turing degrees.

Implicit Conceptions of Infinitary Computation

- α-recursion theory, born from ideas by Takeuti [1], Kreisel and Sacks [2] and some others, in the 1960s.
- E-recursion theory, formulated by Normann [3] in the 1970s.
- These are implicit models of computation because they define what it means to be (relatively) computable without explicitly defining what a computation entails.
- They relate computability to definability by a certain class of formulas, over a specific domain.

Implicit Conceptions of Infinitary Computation

- α-recursion theory, born from ideas by Takeuti [1], Kreisel and Sacks [2] and some others, in the 1960s.
- E-recursion theory, formulated by Normann [3] in the 1970s.
- These are implicit models of computation because they define what it means to be (relatively) computable without explicitly defining what a computation entails.
- They relate computability to definability by a certain class of formulas, over a specific domain.

Implicit Conceptions of Infinitary Computation

- α-recursion theory, born from ideas by Takeuti [1], Kreisel and Sacks [2] and some others, in the 1960s.
- E-recursion theory, formulated by Normann [3] in the 1970s.
- These are implicit models of computation because they define what it means to be (relatively) computable without explicitly defining what a computation entails.
- They relate computability to definability by a certain class of formulas, over a specific domain.

Implicit Conceptions of Infinitary Computation

- α-recursion theory, born from ideas by Takeuti [1], Kreisel and Sacks [2] and some others, in the 1960s.
- E-recursion theory, formulated by Normann [3] in the 1970s.
- These are implicit models of computation because they define what it means to be (relatively) computable without explicitly defining what a computation entails.
- They relate computability to definability by a certain class of formulas, over a specific domain.

Implicit Conceptions of Infinitary Computation

- α-recursion theory, born from ideas by Takeuti [1], Kreisel and Sacks [2] and some others, in the 1960s.
- E-recursion theory, formulated by Normann [3] in the 1970s.
- These are implicit models of computation because they define what it means to be (relatively) computable without explicitly defining what a computation entails.
- They relate computability to definability by a certain class of formulas, over a specific domain.

Outline

In Greater Generality...

- Relative computability is but a form of parametrised definability over some structure.
- Vanilla computability is a special case whereby the parameter in question contains the least amount of information (e.g. using the empty set as parameter).
- From a set-theoretic perspective, computation can thus be thought of as procedural set (even class) generation over a possibly enriched universe of set theory.

In Greater Generality...

- Relative computability is but a form of parametrised definability over some structure.
- Vanilla computability is a special case whereby the parameter in question contains the least amount of information (e.g. using the empty set as parameter).
- From a set-theoretic perspective, computation can thus be thought of as procedural set (even class) generation over a possibly enriched universe of set theory.

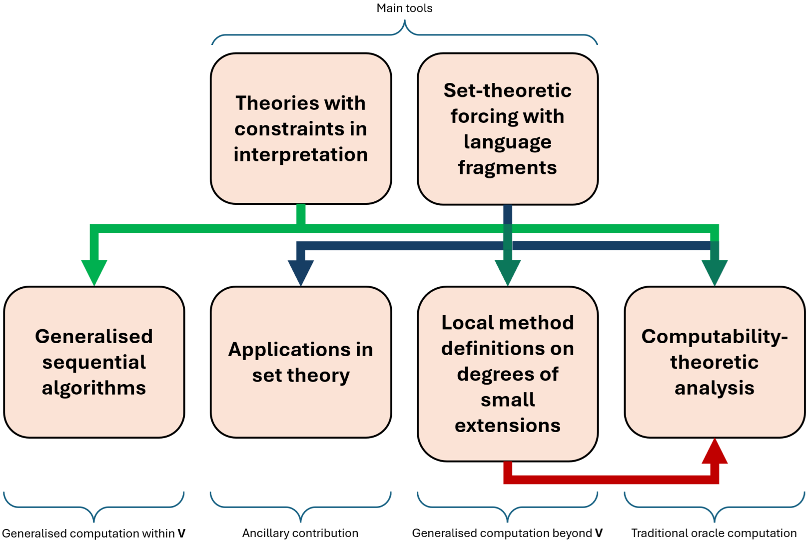
In Greater Generality...

- Relative computability is but a form of parametrised definability over some structure.
- Vanilla computability is a special case whereby the parameter in question contains the least amount of information (e.g. using the empty set as parameter).
- From a set-theoretic perspective, computation can thus be thought of as procedural set (even class) generation over a possibly enriched universe of set theory.

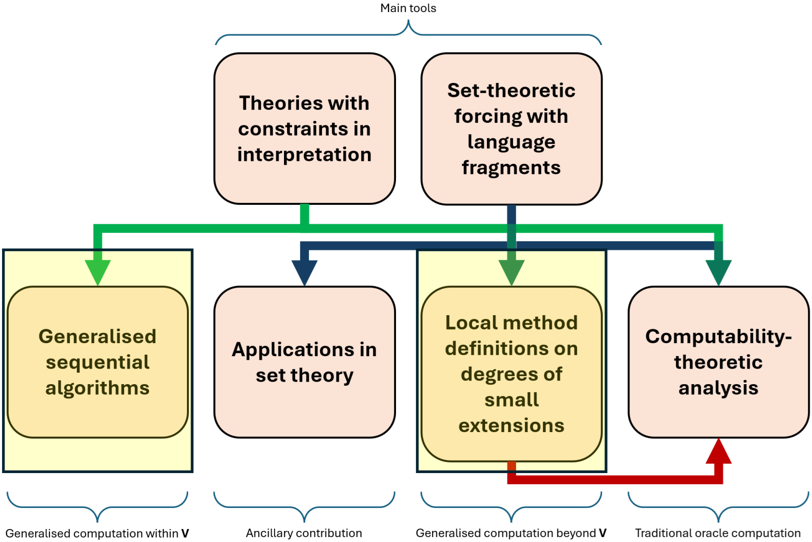
In Greater Generality...

- Relative computability is but a form of parametrised definability over some structure.
- Vanilla computability is a special case whereby the parameter in question contains the least amount of information (e.g. using the empty set as parameter).
- From a set-theoretic perspective, computation can thus be thought of as procedural set (even class) generation over a possibly enriched universe of set theory.

Overview Chart



Today's Focus



Outline

Upshot

- All reasonable transfinite abstract algorithms are expressible as GSeqAPs.
- Here *reasonable* means satisfying the boundedness and locality conditions.

Upshot

- All reasonable transfinite abstract algorithms are expressible as GSeqAPs.
- Here *reasonable* means satisfying the boundedness and locality conditions.

Upshot

- All reasonable transfinite abstract algorithms are expressible as GSeqAPs.
- Here *reasonable* means satisfying the boundedness and locality conditions.

Outline

1. Introduction
   - Background: Infinitary Computation
   - Background: Formulas as Programs
   - Overview
2. Generalised Computation Within V
   - Generalised Sequential Algorithms
   - Comparisons with Other Notions
3. Generalised Computation Beyond V
   - Degrees of Small Extensions
   - Local Method Definitions

Summary Table

In the following table we compare various relative computability relations when they are restricted to subsets of admissible ordinals α.

Property \ Relation	$\leq_\alpha$	$\preceq_\alpha$	$\leq_\alpha^P$	$\leq_\alpha^{P,s}$
Oracle-analogue?	✓	✓	✗	✗
Transitive?	✓	✗	✓	✓
Upward persistent?	✗	✓	✓	✓
Appears in...	α-recursion	α-computability		

Summary Table

In the following table we compare various relative computability relations when they are restricted to subsets of admissible ordinals α.

Relation / Property	$\leq_\alpha$	$\preceq_\alpha$	$\leq_\alpha^P$	$\leq_\alpha^{P,s}$
Oracle-analogue?	✓	✓	✗	✗
Transitive?	✓	✗	✓	✓
Upward persistent?	✗	✓	✓	✓
Appears in...	α-recursion	α-computability		

Summary Table

In the following table we compare various relative computability relations when they are restricted to subsets of admissible ordinals α.

Relation / Property	$\leq_\alpha$	$\preceq_\alpha$	$\leq_\alpha^P$	$\leq_\alpha^{P,s}$
Oracle-analogue?	✓	✓	✗	✗
Transitive?	✓	✗	✓	✓
Upward persistent?	✗	✓	✓	✓
Appears in...	α-recursion	α-computability		

Outline

Generators over V

- Going further in the direction of generalised computation, can we maintain the definition of constructibility degrees, but swop L for V?
- In essence, we want to group sets outside V based on their power as generators over V.

Generators over V

- Going further in the direction of generalised computation, can we maintain the definition of constructibility degrees, but swop L for V?
- In essence, we want to group sets outside V based on their power as generators over V.

Generators over V

- Going further in the direction of generalised computation, can we maintain the definition of constructibility degrees, but swop L for V?
- In essence, we want to group sets outside V based on their power as generators over V.

The Meta-theory

- What do we mean by "sets outside V"?
- Step out of V and treat V as a countable transitive model of ZFC (henceforth denoted CTM).
- Look at those sets that can be adjoined to V to give a nice CTM.
- What do we mean by "a nice CTM"?

The Meta-theory

- What do we mean by "sets outside V"?
- Step out of V and treat V as a countable transitive model of ZFC (henceforth denoted CTM).
- Look at those sets that can be adjoined to V to give a nice CTM.
- What do we mean by "a nice CTM"?

The Meta-theory

- What do we mean by "sets outside V"?
- Step out of V and treat V as a countable transitive model of ZFC (henceforth denoted CTM).
- Look at those sets that can be adjoined to V to give a nice CTM.
- What do we mean by "a nice CTM"?

The Meta-theory

- What do we mean by "sets outside V"?
- Step out of V and treat V as a countable transitive model of ZFC (henceforth denoted CTM).
- Look at those sets that can be adjoined to V to give a nice CTM.
- What do we mean by "a nice CTM"?

The Meta-theory

- What do we mean by "sets outside V"?
- Step out of V and treat V as a countable transitive model of ZFC (henceforth denoted CTM).
- Look at those sets that can be adjoined to V to give a nice CTM.
- What do we mean by "a nice CTM"?

Outer Models

- Let U_1 and U_2 be CTMs. U_2 is an outer model of U_1 iff
  - $U_1 \subset U_2$, and
  - $ORD^{U_1} = ORD^{U_2}$.
- The binary relation "being an outer model of" is transitive.

Definition

Let V be a CTM. The *outward multiverse centred at V* is the set

$$\mathbf{M}(V) := \{W : W \text{ is an outer model of } V\}.$$

Outer Models

- Let U_1 and U_2 be CTMs. U_2 is an outer model of U_1 iff
  - $U_1 \subset U_2$, and
  - $ORD^{U_1} = ORD^{U_2}$.
- The binary relation "being an outer model of" is transitive.

Definition

Let V be a CTM. The *outward multiverse centred at V* is the set

$$\mathbf{M}(V) := \{W : W \text{ is an outer model of } V\}.$$

Outer Models

- Let U_1 and U_2 be CTMs. U_2 is an outer model of U_1 iff
  - $U_1 \subset U_2$, and
  - $ORD^{U_1} = ORD^{U_2}$.
- The binary relation "being an outer model of" is transitive.

Definition

Let V be a CTM. The *outward multiverse centred at V* is the set

$$\mathbf{M}(V) := \{W : W \text{ is an outer model of } V\}.$$

Outer Models

- Let U_1 and U_2 be CTMs. U_2 is an outer model of U_1 iff
  - $U_1 \subset U_2$, and
  - $ORD^{U_1} = ORD^{U_2}$.
- The binary relation "being an outer model of" is transitive.

Definition

Let V be a CTM. The *outward multiverse centred at V* is the set

$$\mathbf{M}(V) := \{W : W \text{ is an outer model of } V\}.$$

Outer Models

- Let U_1 and U_2 be CTMs. U_2 is an outer model of U_1 iff
  - $U_1 \subset U_2$, and
  - $ORD^{U_1} = ORD^{U_2}$.
- The binary relation "being an outer model of" is transitive.

Definition

Let V be a CTM. The *outward multiverse centred at V* is the set

$$\mathbf{M}(V) := \{W : W \text{ is an outer model of } V\}.$$

Outer Models

- Let U_1 and U_2 be CTMs. U_2 is an outer model of U_1 iff
  - $U_1 \subset U_2$, and
  - $ORD^{U_1} = ORD^{U_2}$.
- The binary relation "being an outer model of" is transitive.

Definition

Let V be a CTM. The *outward multiverse centred at V* is the set

$$\mathbf{M}(V) := \{W : W \text{ is an outer model of } V\}.$$

Outer Models

- Let U_1 and U_2 be CTMs. U_2 is an outer model of U_1 iff
  - $U_1 \subset U_2$, and
  - $ORD^{U_1} = ORD^{U_2}$.
- The binary relation "being an outer model of" is transitive.

Definition

Let V be a CTM. The *outward multiverse centred at* V is the set

$$\mathbf{M}(V) := \{W : W \text{ is an outer model of } V\}.$$

Small Extensions

Definition (L.)

Let U_1 and U_2 be CTMs. U_2 is a *small extension of* U_1 iff

- U_2 is an outer model of U_1, and
- there is $x \in U_2$ such that U_2 is the smallest outer model of U_1 containing x.

In this case we call x a *generator of* U_2 *over* U_1, and denote U_2 as $U_1[x]$.

The binary relation "being a small extension of" is transitive.

Definition

Let V be a CTM. The *small outward multiverse centred at* V is the set

$$\mathbf{M}_S(V) := \{W : W \text{ is a small extension of } V\}.$$

Small Extensions

Definition (L.)

Let U_1 and U_2 be CTMs. U_2 is a *small extension of U_1* iff

- U_2 is an outer model of U_1, and
- there is $x \in U_2$ such that U_2 is the smallest outer model of U_1 containing x.

In this case we call x a *generator of U_2 over U_1*, and denote U_2 as $U_1[x]$.

The binary relation "being a small extension of" is transitive.

Definition

Let V be a CTM. The *small outward multiverse centred at V* is the set

$$\mathbf{M}_S(V) := \{W : W \text{ is a small extension of } V\}.$$

Small Extensions

Definition (L.)

Let U_1 and U_2 be CTMs. U_2 is a *small extension of* U_1 iff

- U_2 is an outer model of U_1, and
- there is $x \in U_2$ such that U_2 is the smallest outer model of U_1 containing x.

In this case we call x a *generator of* U_2 *over* U_1, and denote U_2 as $U_1[x]$.

The binary relation "being a small extension of" is transitive.

Definition

Let V be a CTM. The *small outward multiverse centred at* V is the set

$$\mathbf{M}_S(V) := \{W : W \text{ is a small extension of } V\}.$$

Small Extensions

Definition (L.)

Let U_1 and U_2 be CTMs. U_2 is a *small extension of* U_1 iff

- U_2 is an outer model of U_1, and
- there is $x \in U_2$ such that U_2 is the smallest outer model of U_1 containing x.

In this case we call x a *generator of* U_2 *over* U_1, and denote U_2 as $U_1[x]$.

The binary relation "being a small extension of" is transitive.

Definition

Let V be a CTM. The *small outward multiverse centred at* V is the set

$$\mathbf{M}_S(V) := \{W : W \text{ is a small extension of } V\}.$$

Small Extensions

Definition (L.)

Let U_1 and U_2 be CTMs. U_2 is a *small extension of* U_1 iff

- U_2 is an outer model of U_1, and
- there is $x \in U_2$ such that U_2 is the smallest outer model of U_1 containing x.

In this case we call x a *generator of* U_2 *over* U_1, and denote U_2 as $U_1[x]$.

The binary relation "being a small extension of" is transitive.

Definition

Let V be a CTM. The *small outward multiverse centred at* V is the set

$$\mathbf{M}_S(V) := \{W : W \text{ is a small extension of } V\}.$$

Small Extensions

Definition (L.)

Let U_1 and U_2 be CTMs. U_2 is a *small extension of* U_1 iff

- U_2 is an outer model of U_1, and
- there is $x \in U_2$ such that U_2 is the smallest outer model of U_1 containing x.

In this case we call x a *generator of* U_2 *over* U_1, and denote U_2 as $U_1[x]$.

The binary relation "being a small extension of" is transitive.

Definition

Let V be a CTM. The *small outward multiverse centred at* V is the set

$$\mathbf{M}_S(V) := \{W : W \text{ is a small extension of } V\}.$$

Small Extensions

Definition (L.)

Let U_1 and U_2 be CTMs. U_2 is a *small extension of* U_1 iff

- U_2 is an outer model of U_1, and
- there is $x \in U_2$ such that U_2 is the smallest outer model of U_1 containing x.

In this case we call x a *generator of* U_2 *over* U_1, and denote U_2 as $U_1[x]$.

The binary relation "being a small extension of" is transitive.

Definition

Let V be a CTM. The *small outward multiverse centred at* V is the set

$$\mathbf{M}_S(V) := \{W : W \text{ is a small extension of } V\}.$$

Small Extensions

Definition (L.)

Let U_1 and U_2 be CTMs. U_2 is a *small extension of* U_1 iff

- U_2 is an outer model of U_1, and
- there is $x \in U_2$ such that U_2 is the smallest outer model of U_1 containing x.

In this case we call x a *generator of* U_2 *over* U_1, and denote U_2 as $U_1[x]$.

The binary relation "being a small extension of" is transitive.

Definition

Let V be a CTM. The *small outward multiverse centred at* V is the set

$$\mathbf{M}_S(V) := \{W : W \text{ is a small extension of } V\}.$$

Assorted Multiversal Properties

Due to Jensen's result on "coding the universe", we have the following theorem.

Theorem (Jensen)

Given a CTM V, $(\mathbf{M}_S(V), \subset)$ is a cofinal subposet of $(\mathbf{M}(V), \subset)$.

Assorted Multiversal Properties

Due to Jensen's result on "coding the universe", we have the following theorem.

Theorem (Jensen)

Given a CTM V, $(\mathbf{M}_S(V), \subset)$ is a cofinal subposet of $(\mathbf{M}(V), \subset)$.

Assorted Multiversal Properties

Due to Jensen's result on "coding the universe", we have the following theorem.

Theorem (Jensen)

Given a CTM V, $(\mathbf{M}_S(V), \subset)$ is a cofinal subposet of $(\mathbf{M}(V), \subset)$.

Assorted Multiversal Properties

The next proposition is easy to see.

Proposition

Let V be a CTM. Then

$$\mathbf{M}_S(V) = \{V[x] : x \in \bigcup \mathbf{M}(V) \cap \bigcup \{\mathscr{P}(\alpha) : \alpha \in ORD^V\}\}.$$

Assorted Multiversal Properties

The next proposition is easy to see.

Proposition

Let V be a CTM. Then

$$\mathbf{M}_S(V) = \{V[x] : x \in \bigcup \mathbf{M}(V) \cap \bigcup \{\mathscr{P}(\alpha) : \alpha \in ORD^V\}\}.$$

Assorted Multiversal Properties

The next proposition is easy to see.

Proposition

Let V be a CTM. Then

$$\mathbf{M}_S(V) = \{V[x] : x \in \bigcup \mathbf{M}(V) \cap \bigcup \{\mathscr{P}(\alpha) : \alpha \in ORD^V\}\}.$$

Assorted Multiversal Properties

The next proposition is easy to see.

Proposition

Let V be a CTM. Then

$$\mathbf{M}_S(V) = \{V[x] : x \in \bigcup \mathbf{M}(V) \cap \bigcup \{\mathscr{P}(\alpha) : \alpha \in ORD^V\}\}.$$

Assorted Multiversal Properties

- Given a CTM V, we want to define a degree structure on generators of small extensions of V, analogous to the constructibility degrees on sets in V.
- By the previous proposition, it suffices to consider equivalence classes arising from the natural reducibility relation on

$$\mathscr{G}(V) := \bigcup \mathbf{M}(V) \cap \bigcup \{\mathscr{P}(\alpha) : \alpha \in ORD^V\}.$$

Assorted Multiversal Properties

- Given a CTM V, we want to define a degree structure on generators of small extensions of V, analogous to the constructibility degrees on sets in V.
- By the previous proposition, it suffices to consider equivalence classes arising from the natural reducibility relation on

$$\mathscr{G}(V) := \bigcup \mathbf{M}(V) \cap \bigcup \{\mathscr{P}(\alpha) : \alpha \in ORD^V\}.$$

Assorted Multiversal Properties

- Given a CTM V, we want to define a degree structure on generators of small extensions of V, analogous to the constructibility degrees on sets in V.
- By the previous proposition, it suffices to consider equivalence classes arising from the natural reducibility relation on

$$\mathscr{G}(V) := \bigcup \mathbf{M}(V) \cap \bigcup \{\mathscr{P}(\alpha) : \alpha \in ORD^V\}.$$

Assorted Multiversal Properties

- Given a CTM V, we want to define a degree structure on generators of small extensions of V, analogous to the constructibility degrees on sets in V.
- By the previous proposition, it suffices to consider equivalence classes arising from the natural reducibility relation on

$$\mathscr{G}(V) := \bigcup \mathbf{M}(V) \cap \bigcup \{\mathscr{P}(\alpha) : \alpha \in ORD^V\}.$$

Degrees over V

- Specifically, define the binary relation $\leq^S$ on $\mathscr{G}(V)$ by

$$x \leq^S y \iff V[x] \subset V[y].$$

- $\leq^S$ partially orders $\mathscr{G}(V)$, so we can define the equivalence relation $\equiv^S$ the usual way.
- Call $(\mathscr{G} / \equiv^S, \leq^S / \equiv^S)$ the *degrees of small extensions of V* with its standard partial ordering.
- Observe that

$$(\{\text{small extensions of } V\}, \subset) \cong (\mathscr{G} / \equiv^S, \leq^S / \equiv^S),$$

so "small extension(s)" and "degree(s) of small extensions" are often used interchangeably.

Degrees over V

- Specifically, define the binary relation $\leq^S$ on $\mathscr{G}(V)$ by

$$x \leq^S y \iff V[x] \subset V[y].$$

- $\leq^S$ partially orders $\mathscr{G}(V)$, so we can define the equivalence relation $\equiv^S$ the usual way.
- Call $(\mathscr{G} / \equiv^S, \leq^S / \equiv^S)$ the *degrees of small extensions of* V with its standard partial ordering.
- Observe that

$$(\{\text{small extensions of } V\}, \subset) \cong (\mathscr{G} / \equiv^S, \leq^S / \equiv^S),$$

so "small extension(s)" and "degree(s) of small extensions" are often used interchangeably.

Degrees over V

- Specifically, define the binary relation $\leq^S$ on $\mathscr{G}(V)$ by

$$x \leq^S y \iff V[x] \subset V[y].$$

- $\leq^S$ partially orders $\mathscr{G}(V)$, so we can define the equivalence relation $\equiv^S$ the usual way.
- Call $(\mathscr{G} / \equiv^S, \leq^S / \equiv^S)$ the *degrees of small extensions of V* with its standard partial ordering.
- Observe that

$$(\{\text{small extensions of } V\}, \subset) \cong (\mathscr{G} / \equiv^S, \leq^S / \equiv^S),$$

so "small extension(s)" and "degree(s) of small extensions" are often used interchangeably.

Degrees over V

- Specifically, define the binary relation $\leq^S$ on $\mathscr{G}(V)$ by

$$x \leq^S y \iff V[x] \subset V[y].$$

- $\leq^S$ partially orders $\mathscr{G}(V)$, so we can define the equivalence relation $\equiv^S$ the usual way.
- Call $(\mathscr{G} / \equiv^S, \leq^S / \equiv^S)$ the *degrees of small extensions of* V with its standard partial ordering.
- Observe that

$$(\{\text{small extensions of } V\}, \subset) \cong (\mathscr{G} / \equiv^S, \leq^S / \equiv^S),$$

so "small extension(s)" and "degree(s) of small extensions" are often used interchangeably.

Degrees over V

- Specifically, define the binary relation $\leq^S$ on $\mathscr{G}(V)$ by

$$x \leq^S y \iff V[x] \subset V[y].$$

- $\leq^S$ partially orders $\mathscr{G}(V)$, so we can define the equivalence relation $\equiv^S$ the usual way.
- Call $(\mathscr{G} / \equiv^S, \leq^S / \equiv^S)$ the *degrees of small extensions of V* with its standard partial ordering.
- Observe that

$$(\{\text{small extensions of } V\}, \subset) \cong (\mathscr{G} / \equiv^S, \leq^S / \equiv^S),$$

so "small extension(s)" and "degree(s) of small extensions" are often used interchangeably.

Degrees over V

- Specifically, define the binary relation $\leq^S$ on $\mathscr{G}(V)$ by

$$x \leq^S y \iff V[x] \subset V[y].$$

- $\leq^S$ partially orders $\mathscr{G}(V)$, so we can define the equivalence relation $\equiv^S$ the usual way.
- Call $(\mathscr{G} / \equiv^S, \leq^S / \equiv^S)$ the *degrees of small extensions of V* with its standard partial ordering.
- Observe that

$$(\{\text{small extensions of } V\}, \subset) \cong (\mathscr{G} / \equiv^S, \leq^S / \equiv^S),$$

so "small extension(s)" and "degree(s) of small extensions" are often used interchangeably.

Degrees over V

- Specifically, define the binary relation $\leq^S$ on $\mathscr{G}(V)$ by

$$x \leq^S y \iff V[x] \subset V[y].$$

- $\leq^S$ partially orders $\mathscr{G}(V)$, so we can define the equivalence relation $\equiv^S$ the usual way.
- Call $(\mathscr{G}/\equiv^S, \leq^S/\equiv^S)$ the *degrees of small extensions of V* with its standard partial ordering.
- Observe that

$$(\{\text{small extensions of } V\}, \subset) \cong (\mathscr{G}/\equiv^S, \leq^S/\equiv^S),$$

so "small extension(s)" and "degree(s) of small extensions" are often used interchangeably.

Drawing Parallels

Drawing Parallels

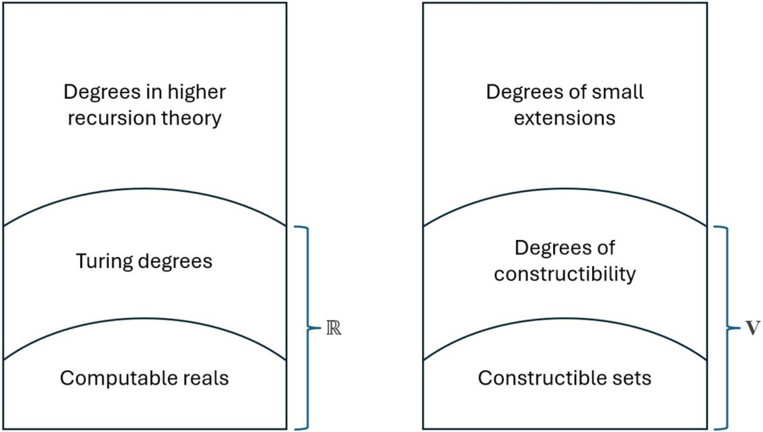


Figure: comparison between conventional notions of relative computability (left) and our generalised notions (right).

Multiverse of Generic Extensions

Definition

Let V be a CTM. The *outward generic multiverse centred at V* is the set

$$\mathbf{M}_F(V) := \{W : W \text{ is a forcing extension of } V\}.$$

The following fact is basic.

Fact

Every generic extension is a small extension. Equivalently,

$$\mathbf{M}_F(V) \subset \mathbf{M}_S(V).$$

Multiverse of Generic Extensions

Definition

Let V be a CTM. The *outward generic multiverse centred at* V is the set

$$\mathbf{M}_F(V) := \{W : W \text{ is a forcing extension of } V\}.$$

The following fact is basic.

Fact

Every generic extension is a small extension. Equivalently,

$$\mathbf{M}_F(V) \subset \mathbf{M}_S(V).$$

Multiverse of Generic Extensions

Definition

Let V be a CTM. The *outward generic multiverse centred at* V is the set

$$\mathbf{M}_F(V) := \{W : W \text{ is a forcing extension of } V\}.$$

The following fact is basic.

Fact

Every generic extension is a small extension. Equivalently,

$$\mathbf{M}_F(V) \subset \mathbf{M}_S(V).$$

Multiverse of Generic Extensions

Definition

Let V be a CTM. The *outward generic multiverse centred at* V is the set

$$\mathbf{M}_F(V) := \{W : W \text{ is a forcing extension of } V\}.$$

The following fact is basic.

Fact

Every generic extension is a small extension. Equivalently,

$$\mathbf{M}_F(V) \subset \mathbf{M}_S(V).$$

Multiverse of Generic Extensions

Definition

Let V be a CTM. The *outward generic multiverse centred at* V is the set

$$\mathbf{M}_F(V) := \{W : W \text{ is a forcing extension of } V\}.$$

The following fact is basic.

Fact

Every generic extension is a small extension. Equivalently,

$$\mathbf{M}_F(V) \subset \mathbf{M}_S(V).$$

Multiverse of Generic Extensions

The following is a rephrasing of a well-known result on intermediate models of forcing extensions.

Fact

$\mathbf{M}_F(V)$ is downward-closed in $\mathbf{M}(V)$, and thus also in $\mathbf{M}_S(V)$.

On the other hand, the next fact can be derived from arguments using class forcing.

Fact

Given a CTM V, $(\mathbf{M}_S(V) \setminus \mathbf{M}_F(V), \subset)$ is a cofinal subposet of $(\mathbf{M}_S(V), \subset)$.

Multiverse of Generic Extensions

The following is a rephrasing of a well-known result on intermediate models of forcing extensions.

Fact

$\mathbf{M}_F(V)$ is downward-closed in $\mathbf{M}(V)$, and thus also in $\mathbf{M}_S(V)$.

On the other hand, the next fact can be derived from arguments using class forcing.

Fact

Given a CTM V, $(\mathbf{M}_S(V) \setminus \mathbf{M}_F(V), \subset)$ is a cofinal subposet of $(\mathbf{M}_S(V), \subset)$.

Multiverse of Generic Extensions

The following is a rephrasing of a well-known result on intermediate models of forcing extensions.

Fact

$\mathbf{M}_F(V)$ is downward-closed in $\mathbf{M}(V)$, and thus also in $\mathbf{M}_S(V)$.

On the other hand, the next fact can be derived from arguments using class forcing.

Fact

Given a CTM V, $(\mathbf{M}_S(V) \setminus \mathbf{M}_F(V), \subset)$ is a cofinal subposet of $(\mathbf{M}_S(V), \subset)$.

Multiverse of Generic Extensions

The following is a rephrasing of a well-known result on intermediate models of forcing extensions.

Fact

$\mathbf{M}_F(V)$ is downward-closed in $\mathbf{M}(V)$, and thus also in $\mathbf{M}_S(V)$.

On the other hand, the next fact can be derived from arguments using class forcing.

Fact

Given a CTM V, $(\mathbf{M}_S(V) \setminus \mathbf{M}_F(V), \subset)$ is a cofinal subposet of $(\mathbf{M}_S(V), \subset)$.

Multiverse of Generic Extensions

The following is a rephrasing of a well-known result on intermediate models of forcing extensions.

Fact

$\mathbf{M}_F(V)$ is downward-closed in $\mathbf{M}(V)$, and thus also in $\mathbf{M}_S(V)$.

On the other hand, the next fact can be derived from arguments using class forcing.

Fact

Given a CTM V, $(\mathbf{M}_S(V) \setminus \mathbf{M}_F(V), \subset)$ is a cofinal subposet of $(\mathbf{M}_S(V), \subset)$.

Accessibility of Forcing

- The previous two facts tell us there are many objects inaccessible by forcing.
- Do these objects have "local first-order properties" not shared by any set in any forcing extension?

Accessibility of Forcing

- The previous two facts tell us there are many objects inaccessible by forcing.
- Do these objects have "local first-order properties" not shared by any set in any forcing extension?

Accessibility of Forcing

- The previous two facts tell us there are many objects inaccessible by forcing.
- Do these objects have "local first-order properties" not shared by any set in any forcing extension?

Outline

Referring to Small Extensions in V

- Often it is useful to refer to small extensions of V within V.
- Such references in general cannot isolate any non-trivial small extension.
- Think of them as descriptions in V picking out sets of small extensions of V, when evaluated outside V.
- We want the evaluations of these descriptions to be reasonably absolute.
- A straightforward way to ensure absoluteness is to make evaluations local to the parameters given in the descriptions.

Referring to Small Extensions in V

- Often it is useful to refer to small extensions of V within V.
- Such references in general cannot isolate any non-trivial small extension.
- Think of them as descriptions in V picking out sets of small extensions of V, when evaluated outside V.
- We want the evaluations of these descriptions to be reasonably absolute.
- A straightforward way to ensure absoluteness is to make evaluations local to the parameters given in the descriptions.

Referring to Small Extensions in V

- Often it is useful to refer to small extensions of V within V.
- Such references in general cannot isolate any non-trivial small extension.
- Think of them as descriptions in V picking out sets of small extensions of V, when evaluated outside V.
- We want the evaluations of these descriptions to be reasonably absolute.
- A straightforward way to ensure absoluteness is to make evaluations local to the parameters given in the descriptions.

Referring to Small Extensions in V

- Often it is useful to refer to small extensions of V within V.
- Such references in general cannot isolate any non-trivial small extension.
- Think of them as descriptions in V picking out sets of small extensions of V, when evaluated outside V.
- We want the evaluations of these descriptions to be reasonably absolute.
- A straightforward way to ensure absoluteness is to make evaluations local to the parameters given in the descriptions.

Referring to Small Extensions in V

- Often it is useful to refer to small extensions of V within V.
- Such references in general cannot isolate any non-trivial small extension.
- Think of them as descriptions in V picking out sets of small extensions of V, when evaluated outside V.
- We want the evaluations of these descriptions to be reasonably absolute.
- A straightforward way to ensure absoluteness is to make evaluations local to the parameters given in the descriptions.

Referring to Small Extensions in V

- Often it is useful to refer to small extensions of V within V.
- Such references in general cannot isolate any non-trivial small extension.
- Think of them as descriptions in V picking out sets of small extensions of V, when evaluated outside V.
- We want the evaluations of these descriptions to be reasonably absolute.
- A straightforward way to ensure absoluteness is to make evaluations local to the parameters given in the descriptions.

Referring to Small Extensions in V

- *Theories with constraints in Interpretation* (*TCIs*), and their models, are a formalisation of this idea.
- TCIs were used as a convenient means of defining state spaces of restricted abstract state machines.
- They also naturally capture the intuition behind forcing.
- A TCI in V describes potential generators of small extensions of V.
- Models of this TCI in $\mathbf{M}_S(V)$ carve out the set of small extensions it describes.

Referring to Small Extensions in V

- *Theories with constraints in Interpretation* (*TCIs*), and their models, are a formalisation of this idea.
- TCIs were used as a convenient means of defining state spaces of restricted abstract state machines.
- They also naturally capture the intuition behind forcing.
- A TCI in V describes potential generators of small extensions of V.
- Models of this TCI in $\mathbf{M}_S(V)$ carve out the set of small extensions it describes.

Referring to Small Extensions in V

- *Theories with constraints in Interpretation* (*TCIs*), and their models, are a formalisation of this idea.
- TCIs were used as a convenient means of defining state spaces of restricted abstract state machines.
- They also naturally capture the intuition behind forcing.
- A TCI in V describes potential generators of small extensions of V.
- Models of this TCI in $\mathbf{M}_S(V)$ carve out the set of small extensions it describes.

Referring to Small Extensions in V

- *Theories with constraints in Interpretation* (*TCIs*), and their models, are a formalisation of this idea.
- TCIs were used as a convenient means of defining state spaces of restricted abstract state machines.
- They also naturally capture the intuition behind forcing.
- A TCI in V describes potential generators of small extensions of V.
- Models of this TCI in $\mathbf{M}_S(V)$ carve out the set of small extensions it describes.

Referring to Small Extensions in V

- *Theories with constraints in Interpretation* (*TCIs*), and their models, are a formalisation of this idea.
- TCIs were used as a convenient means of defining state spaces of restricted abstract state machines.
- They also naturally capture the intuition behind forcing.
- A TCI in V describes potential generators of small extensions of V.
- Models of this TCI in $\mathbf{M}_S(V)$ carve out the set of small extensions it describes.

Referring to Small Extensions in V

- *Theories with constraints in Interpretation* (*TCIs*), and their models, are a formalisation of this idea.
- TCIs were used as a convenient means of defining state spaces of restricted abstract state machines.
- They also naturally capture the intuition behind forcing.
- A TCI in V describes potential generators of small extensions of V.
- Models of this TCI in $\mathbf{M}_S(V)$ carve out the set of small extensions it describes.

Theories with Constraints in Interpretation

- TCIs provide bounds to the interpretation of a theory T over a signature σ.
- A TCI $\mathscr{T}$ expands on T by specifying additional requirements on a possible model $\mathfrak{M}$ of T.
- These requirements include
  - a set upper bound (under $\subset$) to the base set M of $\mathfrak{M}$,
  - a set upper bound to $\dot{X}^{\mathfrak{M}}$, for each $\dot{X} \in \sigma$, and
  - whether each $\dot{X}^{\mathfrak{M}}$ depends entirely on its upper bound and M.

Theories with Constraints in Interpretation

- TCIs provide bounds to the interpretation of a theory T over a signature σ.
- A TCI $\mathscr{T}$ expands on T by specifying additional requirements on a possible model $\mathfrak{M}$ of T.
- These requirements include
  - a set upper bound (under $\subset$) to the base set M of $\mathfrak{M}$,
  - a set upper bound to $\dot{X}^{\mathfrak{M}}$, for each $\dot{X} \in \sigma$, and
  - whether each $\dot{X}^{\mathfrak{M}}$ depends entirely on its upper bound and M.

Theories with Constraints in Interpretation

- TCIs provide bounds to the interpretation of a theory T over a signature σ.
- A TCI $\mathscr{T}$ expands on T by specifying additional requirements on a possible model $\mathfrak{M}$ of T.
- These requirements include
  - a set upper bound (under $\subset$) to the base set M of $\mathfrak{M}$,
  - a set upper bound to $\dot{X}^{\mathfrak{M}}$, for each $\dot{X} \in \sigma$, and
  - whether each $\dot{X}^{\mathfrak{M}}$ depends entirely on its upper bound and M.

Theories with Constraints in Interpretation

- TCIs provide bounds to the interpretation of a theory T over a signature σ.
- A TCI $\mathscr{T}$ expands on T by specifying additional requirements on a possible model $\mathfrak{M}$ of T.
- These requirements include
  - a set upper bound (under $\subset$) to the base set M of $\mathfrak{M}$,
  - a set upper bound to $\dot{X}^{\mathfrak{M}}$, for each $\dot{X} \in \sigma$, and
  - whether each $\dot{X}^{\mathfrak{M}}$ depends entirely on its upper bound and M.

Theories with Constraints in Interpretation

- TCIs provide bounds to the interpretation of a theory T over a signature σ.
- A TCI $\mathscr{T}$ expands on T by specifying additional requirements on a possible model $\mathfrak{M}$ of T.
- These requirements include
  - a set upper bound (under $\subset$) to the base set M of $\mathfrak{M}$,
  - a set upper bound to $\dot{X}^{\mathfrak{M}}$, for each $\dot{X} \in \sigma$, and
  - whether each $\dot{X}^{\mathfrak{M}}$ depends entirely on its upper bound and M.

Theories with Constraints in Interpretation

- TCIs provide bounds to the interpretation of a theory T over a signature σ.
- A TCI $\mathscr{T}$ expands on T by specifying additional requirements on a possible model $\mathfrak{M}$ of T.
- These requirements include
  - a set upper bound (under $\subset$) to the base set M of $\mathfrak{M}$,
  - a set upper bound to $\dot{X}^{\mathfrak{M}}$, for each $\dot{X} \in \sigma$, and
  - whether each $\dot{X}^{\mathfrak{M}}$ depends entirely on its upper bound and M.

Theories with Constraints in Interpretation

- TCIs provide bounds to the interpretation of a theory T over a signature σ.
- A TCI $\mathscr{T}$ expands on T by specifying additional requirements on a possible model $\mathfrak{M}$ of T.
- These requirements include
  - a set upper bound (under $\subset$) to the base set M of $\mathfrak{M}$,
  - a set upper bound to $\dot{X}^{\mathfrak{M}}$, for each $\dot{X} \in \sigma$, and
  - whether each $\dot{X}^{\mathfrak{M}}$ depends entirely on its upper bound and M.

Consistency of TCIs

Definition (L.)

A TCI is *consistent* iff it has a model in some outer model of V.

By examining a suitable forcing notion in V, we can decide if a TCI in V is consistent.

Proposition

Let $\mathscr{T}$ be a TCI. Then for all sufficiently large cardinals λ,

$$\Vdash_{Col(\omega,\lambda)} \exists \mathfrak{M} \text{ (“}\mathfrak{M} \textit{ is a model of } \mathscr{T}\text{”)} \iff \mathscr{T} \textit{ is consistent.}$$

Consistency of TCIs

Definition (L.)

A TCI is *consistent* iff it has a model in some outer model of V.

By examining a suitable forcing notion in V, we can decide if a TCI in V is consistent.

Proposition

Let $\mathscr{T}$ be a TCI. Then for all sufficiently large cardinals λ,

$$\Vdash_{Col(\omega,\lambda)} \exists \mathfrak{M} \, (\text{“}\mathfrak{M} \textit{ is a model of } \mathscr{T}\text{”}) \iff \mathscr{T} \textit{ is consistent.}$$

Consistency of TCIs

Definition (L.)

A TCI is *consistent* iff it has a model in some outer model of V.

By examining a suitable forcing notion in V, we can decide if a TCI in V is consistent.

Proposition

Let $\mathscr{T}$ be a TCI. Then for all sufficiently large cardinals λ,

$$\Vdash_{Col(\omega,\lambda)} \exists \mathfrak{M} \text{ (``}\mathfrak{M} \textit{ is a model of } \mathscr{T}\text{'')} \iff \mathscr{T} \textit{ is consistent.}$$

Consistency of TCIs

Definition (L.)

A TCI is *consistent* iff it has a model in some outer model of V.

By examining a suitable forcing notion in V, we can decide if a TCI in V is consistent.

Proposition

Let $\mathscr{T}$ be a TCI. Then for all sufficiently large cardinals λ,

$$\Vdash_{Col(\omega,\lambda)} \exists \mathfrak{M} \text{ (“}\mathfrak{M} \textit{ is a model of } \mathscr{T}\text{”)} \iff \mathscr{T} \textit{ is consistent.}$$

Consistency of TCIs

Definition (L.)

A TCI is *consistent* iff it has a model in some outer model of V.

By examining a suitable forcing notion in V, we can decide if a TCI in V is consistent.

Proposition

Let $\mathscr{T}$ be a TCI. Then for all sufficiently large cardinals λ,

$$\Vdash_{Col(\omega,\lambda)} \exists \mathfrak{M} \text{ (“}\mathfrak{M} \textit{ is a model of } \mathscr{T}\text{”)} \iff \mathscr{T} \textit{ is consistent.}$$

Local Method Definitions

Definition (L.)

A *local method definition of V* is a non-empty class of TCIs definable in V.

- In the meta-theory, define a function Eval^V from the set of TCIs $\mathscr{T} \in V$ into the set of small extensions of V, such that

$$\mathrm{Eval}^V(\mathscr{T}) = \{V[\mathfrak{M}] \in \mathbf{M}_S(V) : \mathfrak{M} \text{ is a model of } \mathscr{T}\}.$$

- Each local method definition then picks out multiple sets of small extensions of V.

Local Method Definitions

Definition (L.)

A *local method definition of* V is a non-empty class of TCIs definable in V.

- In the meta-theory, define a function Eval^V from the set of TCIs $\mathscr{T} \in V$ into the set of small extensions of V, such that

$$\mathrm{Eval}^V(\mathscr{T}) = \{V[\mathfrak{M}] \in \mathbf{M}_S(V) : \mathfrak{M} \text{ is a model of } \mathscr{T}\}.$$

- Each local method definition then picks out multiple sets of small extensions of V.

Local Method Definitions

Definition (L.)

A *local method definition of* V is a non-empty class of TCIs definable in V.

- In the meta-theory, define a function Eval^V from the set of TCIs $\mathscr{T} \in V$ into the set of small extensions of V, such that

$$\mathrm{Eval}^V(\mathscr{T}) = \{V[\mathfrak{M}] \in \mathbf{M}_S(V) : \mathfrak{M} \text{ is a model of } \mathscr{T}\}.$$

- Each local method definition then picks out multiple sets of small extensions of V.

Local Method Definitions

Definition (L.)

A *local method definition of* V is a non-empty class of TCIs definable in V.

- In the meta-theory, define a function Eval^V from the set of TCIs $\mathscr{T} \in V$ into the set of small extensions of V, such that

$$\mathrm{Eval}^V(\mathscr{T}) = \{V[\mathfrak{M}] \in \mathbf{M}_S(V) : \mathfrak{M} \text{ is a model of } \mathscr{T}\}.$$

- Each local method definition then picks out multiple sets of small extensions of V.

Local Method Definitions

Definition (L.)

A *local method definition of* V is a non-empty class of TCIs definable in V.

- In the meta-theory, define a function Eval^V from the set of TCIs $\mathscr{T} \in V$ into the set of small extensions of V, such that

$$\mathrm{Eval}^V(\mathscr{T}) = \{V[\mathfrak{M}] \in \mathbf{M}_S(V) : \mathfrak{M} \text{ is a model of } \mathscr{T}\}.$$

- Each local method definition then picks out multiple sets of small extensions of V.

Comparing Local Methods

Definition (L.)

If X and Y are local method definitions of V, $X \leq^M Y$ denotes the statement

"there is a function $F : X \longrightarrow Y$ definable in V such that $\emptyset \neq \mathrm{Eval}^V(F(\mathscr{T})) \subset \mathrm{Eval}^V(\mathscr{T})$ for all consistent $\mathscr{T} \in X$".

- Intuitively, $X \leq^M Y$ if V can see that Y provides non-trivial refinements to all consistent descriptions in X.
- $\leq^M$ partially orders local method definitions, so we can define the equivalence relation $\equiv^M$ the usual way.

Observation

Let X, Y be local method definitions. If $X \subset Y$, then $X \leq^M Y$.

Comparing Local Methods

Definition (L.)

If X and Y are local method definitions of V, $X \leq^M Y$ denotes the statement

"there is a function $F : X \longrightarrow Y$ definable in V such that $\emptyset \neq \mathrm{Eval}^V(F(\mathscr{T})) \subset \mathrm{Eval}^V(\mathscr{T})$ for all consistent $\mathscr{T} \in X$".

- Intuitively, $X \leq^M Y$ if V can see that Y provides non-trivial refinements to all consistent descriptions in X.
- $\leq^M$ partially orders local method definitions, so we can define the equivalence relation $\equiv^M$ the usual way.

Observation

Let X, Y be local method definitions. If $X \subset Y$, then $X \leq^M Y$.

Comparing Local Methods

Definition (L.)

If X and Y are local method definitions of V, $X \leq^M Y$ denotes the statement

> "there is a function $F : X \longrightarrow Y$ definable in V such that $\emptyset \neq \mathrm{Eval}^V(F(\mathscr{T})) \subset \mathrm{Eval}^V(\mathscr{T})$ for all consistent $\mathscr{T} \in X$".

- Intuitively, $X \leq^M Y$ if V can see that Y provides non-trivial refinements to all consistent descriptions in X.
- $\leq^M$ partially orders local method definitions, so we can define the equivalence relation $\equiv^M$ the usual way.

Observation

Let X, Y be local method definitions. If $X \subset Y$, then $X \leq^M Y$.

Comparing Local Methods

Definition (L.)

If X and Y are local method definitions of V, $X \leq^M Y$ denotes the statement

"there is a function $F : X \longrightarrow Y$ definable in V such that $\emptyset \neq \text{Eval}^V(F(\mathscr{T})) \subset \text{Eval}^V(\mathscr{T})$ for all consistent $\mathscr{T} \in X$".

- Intuitively, $X \leq^M Y$ if V can see that Y provides non-trivial refinements to all consistent descriptions in X.
- $\leq^M$ partially orders local method definitions, so we can define the equivalence relation $\equiv^M$ the usual way.

Observation

Let X, Y be local method definitions. If $X \subset Y$, then $X \leq^M Y$.

Comparing Local Methods

Definition (L.)

If X and Y are local method definitions of V, $X \leq^M Y$ denotes the statement

"there is a function $F : X \longrightarrow Y$ definable in V such that $\emptyset \neq \mathrm{Eval}^V(F(\mathscr{T})) \subset \mathrm{Eval}^V(\mathscr{T})$ for all consistent $\mathscr{T} \in X$".

- Intuitively, $X \leq^M Y$ if V can see that Y provides non-trivial refinements to all consistent descriptions in X.
- $\leq^M$ partially orders local method definitions, so we can define the equivalence relation $\equiv^M$ the usual way.

Observation

Let X, Y be local method definitions. If $X \subset Y$, then $X \leq^M Y$.

Comparing Local Methods

Definition (L.)

If X and Y are local method definitions of V, $X \leq^M Y$ denotes the statement

"there is a function $F : X \longrightarrow Y$ definable in V such that $\emptyset \neq \mathrm{Eval}^V(F(\mathscr{T})) \subset \mathrm{Eval}^V(\mathscr{T})$ for all consistent $\mathscr{T} \in X$".

- Intuitively, $X \leq^M Y$ if V can see that Y provides non-trivial refinements to all consistent descriptions in X.
- $\leq^M$ partially orders local method definitions, so we can define the equivalence relation $\equiv^M$ the usual way.

Observation

Let X, Y be local method definitions. If $X \subset Y$, then $X \leq^M Y$.

Comparing Local Methods

Comparing Local Methods

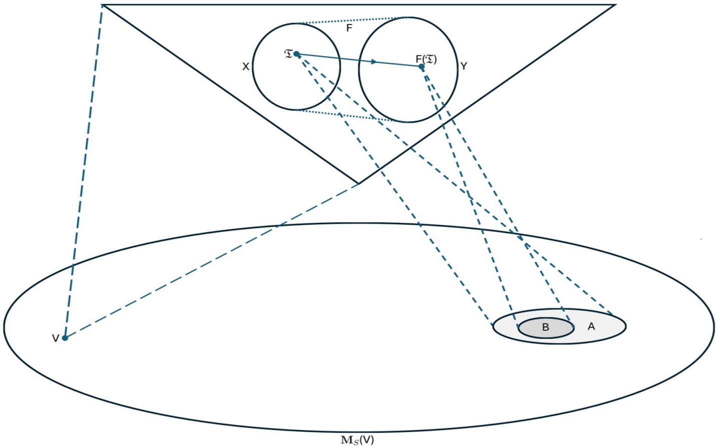


Figure: Visual representation of a function F witnessing $X \leq^M Y$, where X and Y are local method definitions.

The Local Method Hierarchy

- We can also define the complexity of a TCI by just looking at its first-order theory.
- For example, a Σ_n TCI has a first-order theory containing only Σ_n sentences.
- The classes of Σ_n and Π_n TCIs — denoted $\Sigma^{\mathsf{M}}_{\mathsf{n}}$ and $\Pi^{\mathsf{M}}_{\mathsf{n}}$ respectively — then form a hierarchy under the relation $\leq^M$, called the local method hierarchy.

The Local Method Hierarchy

- We can also define the complexity of a TCI by just looking at its first-order theory.
- For example, a Σ_n TCI has a first-order theory containing only Σ_n sentences.
- The classes of Σ_n and Π_n TCIs — denoted $\Sigma^{\mathrm{M}}_{\mathrm{n}}$ and $\Pi^{\mathrm{M}}_{\mathrm{n}}$ respectively — then form a hierarchy under the relation $\leq^M$, called the local method hierarchy.

The Local Method Hierarchy

- We can also define the complexity of a TCI by just looking at its first-order theory.
- For example, a Σ_n TCI has a first-order theory containing only Σ_n sentences.
- The classes of Σ_n and Π_n TCIs — denoted $\Sigma^{\mathsf{M}}_{\mathsf{n}}$ and $\Pi^{\mathsf{M}}_{\mathsf{n}}$ respectively — then form a hierarchy under the relation $\leq^M$, called the local method hierarchy.

The Local Method Hierarchy

- We can also define the complexity of a TCI by just looking at its first-order theory.
- For example, a Σ_n TCI has a first-order theory containing only Σ_n sentences.
- The classes of Σ_n and Π_n TCIs — denoted $\mathsf{\Sigma^M_n}$ and $\mathsf{\Pi^M_n}$ respectively — then form a hierarchy under the relation $\leq^M$, called the local method hierarchy.

The Local Method Hierarchy

Theorem (L.)

Let $1 \leq n < \omega$. Then for every $\mathscr{T} \in \Pi^{\mathsf{M}}_{\mathsf{n+1}}$ there is $\mathscr{T}' \in \Sigma^{\mathsf{M}}_{\mathsf{n}}$ such that

$$\mathrm{Eval}^V(\mathscr{T}) = \mathrm{Eval}^V(\mathscr{T}').$$

Corollary

$\Pi^{\mathsf{M}}_{\mathsf{n+1}} \leq^M \Sigma^{\mathsf{M}}_{\mathsf{n}}$ *for all* $1 \leq n < \omega$.

The Local Method Hierarchy

Theorem (L.)

Let $1 \leq n < \omega$. Then for every $\mathscr{T} \in \Pi^{\mathsf{M}}_{\mathsf{n+1}}$ there is $\mathscr{T}' \in \Sigma^{\mathsf{M}}_{\mathsf{n}}$ such that

$$\mathrm{Eval}^V(\mathscr{T}) = \mathrm{Eval}^V(\mathscr{T}').$$

Corollary

$\Pi^{\mathsf{M}}_{\mathsf{n+1}} \leq^M \Sigma^{\mathsf{M}}_{\mathsf{n}}$ *for all $1 \leq n < \omega$.*

The Local Method Hierarchy

Theorem (L.)

Let $1 \leq n < \omega$. Then for every $\mathscr{T} \in \Pi^{\mathsf{M}}_{\mathsf{n+1}}$ there is $\mathscr{T}' \in \Sigma^{\mathsf{M}}_{\mathsf{n}}$ such that

$$\mathrm{Eval}^V(\mathscr{T}) = \mathrm{Eval}^V(\mathscr{T}').$$

Corollary

$\Pi^{\mathsf{M}}_{\mathsf{n+1}} \leq^M \Sigma^{\mathsf{M}}_{\mathsf{n}}$ *for all* $1 \leq n < \omega$.

The Local Method Hierarchy

Theorem (L.)

Let $1 \leq n < \omega$. Then for every $\mathscr{T} \in \Pi^{\mathsf{M}}_{\mathsf{n+1}}$ there is $\mathscr{T}' \in \Sigma^{\mathsf{M}}_{\mathsf{n}}$ such that

$$\mathrm{Eval}^V(\mathscr{T}) = \mathrm{Eval}^V(\mathscr{T}').$$

Corollary

$\Pi^{\mathsf{M}}_{\mathsf{n+1}} \leq^M \Sigma^{\mathsf{M}}_{\mathsf{n}}$ *for all* $1 \leq n < \omega$.

Set Forcing

- There is an obvious way of representing any forcing notion as a Π_2 TCI.
- Thus set forcing, as a technique of accessing small extensions of V, can be represented by a local method definition, denoted Fg.
- We want to see if Fg fits nicely in the local method hierarchy.
- From what we know so far, $\mathrm{Fg} \leq^M \Sigma_1^{\mathrm{M}}$.

Set Forcing

- There is an obvious way of representing any forcing notion as a Π_2 TCI.
- Thus set forcing, as a technique of accessing small extensions of V, can be represented by a local method definition, denoted Fg.
- We want to see if Fg fits nicely in the local method hierarchy.
- From what we know so far, $\mathrm{Fg} \leq^M \Sigma_1^{\mathrm{M}}$.

Set Forcing

- There is an obvious way of representing any forcing notion as a Π_2 TCI.
- Thus set forcing, as a technique of accessing small extensions of V, can be represented by a local method definition, denoted Fg.
- We want to see if Fg fits nicely in the local method hierarchy.
- From what we know so far, Fg $\leq^M \Sigma_1^{\mathrm{M}}$.

Set Forcing

- There is an obvious way of representing any forcing notion as a Π_2 TCI.
- Thus set forcing, as a technique of accessing small extensions of V, can be represented by a local method definition, denoted Fg.
- We want to see if Fg fits nicely in the local method hierarchy.
- From what we know so far, Fg $\leq^M \Sigma_1^{\mathrm{M}}$.

Set Forcing

- There is an obvious way of representing any forcing notion as a Π_2 TCI.
- Thus set forcing, as a technique of accessing small extensions of V, can be represented by a local method definition, denoted Fg.
- We want to see if Fg fits nicely in the local method hierarchy.
- From what we know so far, $\mathsf{Fg} \leq^M \Sigma_1^{\mathsf{M}}$.

Set Forcing is Σ_1 (is Π_2)

Theorem (L.)

$\mathsf{Fg} \equiv^M \Sigma_1^{\mathsf{M}}$.

Proof Idea.

By adapting our framework for constructing forcing notions to the context of TCIs, we associate with each Π_2 TCI $\mathscr{T}$ a forcing notion $\mathbb{P}(\mathscr{T})$, such that a unique model of $\mathscr{T}$ can be read off every $\mathbb{P}(\mathscr{T})$-generic filter over V.

This allows us to define a class function witnessing $\Pi_2^{\mathsf{M}} \leq^M \mathsf{Fg}$. $\square$

Set Forcing is Σ_1 (is Π_2)

Theorem (L.)

Fg $\equiv^M \Sigma_1^{\mathsf{M}}$.

Proof Idea.

By adapting our framework for constructing forcing notions to the context of TCIs, we associate with each Π_2 TCI $\mathscr{T}$ a forcing notion $\mathbb{P}(\mathscr{T})$, such that a unique model of $\mathscr{T}$ can be read off every $\mathbb{P}(\mathscr{T})$-generic filter over V.

This allows us to define a class function witnessing $\Pi_2^{\mathsf{M}} \leq^M$ Fg. $\square$

Set Forcing is Σ_1 (is Π_2)

Theorem (L.)

$\mathsf{Fg} \equiv^M \Sigma_1^{\mathsf{M}}$.

Proof Idea.

By adapting our framework for constructing forcing notions to the context of TCIs, we associate with each Π_2 TCI $\mathscr{T}$ a forcing notion $\mathbb{P}(\mathscr{T})$, such that a unique model of $\mathscr{T}$ can be read off every $\mathbb{P}(\mathscr{T})$-generic filter over V.

This allows us to define a class function witnessing $\Pi_2^{\mathsf{M}} \leq^M \mathsf{Fg}$. $\square$

Set Forcing is Σ_1 (is Π_2)

Theorem (L.)

$\mathsf{Fg} \equiv^M \Sigma_1^{\mathsf{M}}$.

Proof Idea.

By adapting our framework for constructing forcing notions to the context of TCIs, we associate with each Π_2 TCI $\mathscr{T}$ a forcing notion $\mathbb{P}(\mathscr{T})$, such that a unique model of $\mathscr{T}$ can be read off every $\mathbb{P}(\mathscr{T})$-generic filter over V.

This allows us to define a class function witnessing $\Pi_2^{\mathsf{M}} \leq^M \mathsf{Fg}$. $\square$

Further Results

- We can refine the function witnessing $\Pi_2^{\mathrm{M}} \leq^M \mathrm{Fg}$ by iteratively pruning the $\mathbb{P}(\mathscr{T})$s of atoms.
- There are analogues of the previous theorem in V pertaining to certain countable Π_2 TCIs.
  - Essentially, we can define functions F of low complexity taking these TCIs to reals such that every $F(\mathscr{T})$-1-generic real codes a model of $\mathscr{T}$.

Further Results

- We can refine the function witnessing $\Pi_2^{\mathsf{M}} \leq^M \mathsf{Fg}$ by iteratively pruning the $\mathbb{P}(\mathscr{T})$s of atoms.
- There are analogues of the previous theorem in V pertaining to certain countable Π_2 TCIs.
  - Essentially, we can define functions F of low complexity taking these TCIs to reals such that every $F(\mathscr{T})$-1-generic real codes a model of $\mathscr{T}$.

Further Results

- We can refine the function witnessing $\Pi_2^{\mathsf{M}} \leq^M \mathsf{Fg}$ by iteratively pruning the $\mathbb{P}(\mathscr{T})$s of atoms.
- There are analogues of the previous theorem in V pertaining to certain countable Π_2 TCIs.
  - Essentially, we can define functions F of low complexity taking these TCIs to reals such that every $F(\mathscr{T})$-1-generic real codes a model of $\mathscr{T}$.

Further Results

- We can refine the function witnessing $\Pi_2^{\mathsf{M}} \leq^M \mathsf{Fg}$ by iteratively pruning the $\mathbb{P}(\mathscr{T})$s of atoms.
- There are analogues of the previous theorem in V pertaining to certain countable Π_2 TCIs.
  - Essentially, we can define functions F of low complexity taking these TCIs to reals such that every $F(\mathscr{T})$-1-generic real codes a model of $\mathscr{T}$.

Some Open Questions

(Q1) What structural properties of $(\mathbf{M}_F(L), \subset)$ also hold for $(\mathbf{M}_S(L), \subset)$?

(Q2) For example, is $(\mathbf{M}_S(L), \subset)$ downward directed?

(Q3) Are there $m, n < \omega$ for which $\Sigma^{\mathsf{M}}_{\mathsf{m}} \neq^M \Sigma^{\mathsf{M}}_{\mathsf{n}}$?

(Q4) Is there a TCI $\mathscr{T}$ such that $\{\mathscr{T}\} \not\leq^M \mathsf{Fg}$?

Some Open Questions

(Q1) What structural properties of $(\mathbf{M}_F(L), \subset)$ also hold for $(\mathbf{M}_S(L), \subset)$?

(Q2) For example, is $(\mathbf{M}_S(L), \subset)$ downward directed?

(Q3) Are there $m, n < \omega$ for which $\Sigma^{\mathsf{M}}_{\mathsf{m}} \not\equiv^M \Sigma^{\mathsf{M}}_{\mathsf{n}}$?

(Q4) Is there a TCI $\mathscr{T}$ such that $\{\mathscr{T}\} \not\leq^M \mathrm{Fg}$?

Some Open Questions

(Q1) What structural properties of $(\mathbf{M}_F(L), \subset)$ also hold for $(\mathbf{M}_S(L), \subset)$?

(Q2) For example, is $(\mathbf{M}_S(L), \subset)$ downward directed?

(Q3) Are there $m, n < \omega$ for which $\Sigma^{\mathsf{M}}_{\mathsf{m}} \not\equiv^M \Sigma^{\mathsf{M}}_{\mathsf{n}}$?

(Q4) Is there a TCI $\mathscr{T}$ such that $\{\mathscr{T}\} \not\leq^M \mathrm{Fg}$?

Some Open Questions

(Q1) What structural properties of $(\mathbf{M}_F(L), \subset)$ also hold for $(\mathbf{M}_S(L), \subset)$?

(Q2) For example, is $(\mathbf{M}_S(L), \subset)$ downward directed?

(Q3) Are there $m, n < \omega$ for which $\Sigma^{\mathsf{M}}_{\mathsf{m}} \not\equiv^M \Sigma^{\mathsf{M}}_{\mathsf{n}}$?

(Q4) Is there a TCI $\mathscr{T}$ such that $\{\mathscr{T}\} \not\leq^M \mathsf{Fg}$?

Some Open Questions

(Q1) What structural properties of $(\mathbf{M}_F(L), \subset)$ also hold for $(\mathbf{M}_S(L), \subset)$?

(Q2) For example, is $(\mathbf{M}_S(L), \subset)$ downward directed?

(Q3) Are there $m, n < \omega$ for which $\Sigma^{\mathsf{M}}_{\mathsf{m}} \not\equiv^{M} \Sigma^{\mathsf{M}}_{\mathsf{n}}$?

(Q4) Is there a TCI $\mathscr{T}$ such that $\{\mathscr{T}\} \not\leq^{M} \mathsf{Fg}$?

References I

[1] Gaisi Takeuti. “On the recursive functions of ordinal numbers”. In: *Journal of the Mathematical Society of Japan* 12.2 (1960), pp. 119–128.

[2] Georg Kreisel and Gerald E Sacks. “Metarecursive sets”. In: *The Journal of Symbolic Logic* 30.3 (1965), pp. 318–338.

[3] Dag Normann. “Set recursion”. In: *Studies in Logic and the Foundations of Mathematics*. Vol. 94. Elsevier, 1978, pp. 303–320.

[4] Kurt Gödel. *Collected Works, Volume 2: Publications 1938-1974*. Oxford, England and New York, NY, USA, 1986.

References II

[5] Lenore Blum, Mike Shub, and Steve Smale. "On a theory of computation and complexity over the real numbers: NP-completeness, recursive functions and universal machines". In: *Bulletin of the American Mathematical Society* 21.1 (1989), pp. 1–46.

[6] Joel David Hamkins and Andy Lewis. "Infinite time Turing machines". In: *The Journal of Symbolic Logic* 65.2 (2000), pp. 567–604.

References III

[7] Keng Meng Ng, Nazanin R Tavana, and Yue Yang. "A recursion theoretic foundation of computation over real numbers". In: *Journal of Logic and Computation* 31.7 (July 2021), pp. 1660–1689. ISSN: 0955-792X. DOI: 10.1093/logcom/exab042. eprint: https://academic.oup.com/logcom/article-pdf/31/7/1660/40820416/exab042.pdf. URL: https://doi.org/10.1093/logcom/exab042.

Thank You!